FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Student Name	Gabriel A. Cuesta
Section:	AN43
Professor:	Crisola G. Tan
Date:	October 25, 2026

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge. Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
    (Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule
3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]
Rule-Based Reasoning Actions: - Turn on the aircon - Bring umbrella

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [
    {"problem": {"won't turn on": True, "noise": False},
     "solution": "Check power cable."}
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?
   - I think that RBR was easier to implement because RBR uses an IF-THEN cases where rules are already established and only implement solutions or actions based on that rule alone. On the other hand, CBR needed a function that checks similarity to past cases and tries to adapt from the closest case in terms of similarity which can be ineffective or inaccurate.
2. Which is more flexible in solving unstructured problems?
   - CBR. Since RBR uses rules, which are in a way, absolute, CBR uses past events and try to find anything close or related to the ongoing case/problem, meaning that the problem didn't have to be an exact replica of past cases but it still could help as long as there's some similarity even in the smallest aspect.
3. Which is better for situations with continuously changing data?
   - CBR, because this kind of system uses the past cases to try to relate to the current problem and attempts to create a solution, adapting to newer problems without direct changes from the programmer. RBR uses fixed rules and if a change is needed, the programmer still has to manually change it themselves.
4. Examples of real-world applications for both.

- Using both systems in the medical field, RBR can be used on how much dosage or drug should be used on a patient. For CBR, it can be used to relate patients to past cases with similar symptoms to give the nurse/doctor a better idea on how to act especially in case of unfamiliar or uncommon symptoms.

Your Code - Laboratory Tasks ⇩

Building knowledge

```
What kind of medical problem are you experiencing?
  [1] Fever / flu-like illness
  [2] Cough or breathing difficulty
  [3] Stomach / digestive problem
  [4] Headache
  [5] Skin rash / allergy
Your choice: 2

Are you having trouble breathing?
  [1] Yes
  [2] No
Your choice: 2

Do you have a sore throat?
  [1] Yes
  [2] No
Your choice: 2

How is your body temperature?
  [1] Normal (below 37.5 C)
  [2] Mild fever (37.5 - 38.9 C)
  [3] High fever (39 C or above)
Your choice: 1

How severe are the symptoms?
  [1] Mild
  [2] Moderate
  [3] Severe
Your choice: 1

How long have you had these symptoms?
  [1] Less than 2 days
  [2] 2 to 5 days
  [3] More than 5 days
Your choice: 1

Knowledge Base (facts):
  vitals: {'temperature': 36.8, 'has_fever': False}
  symptoms: {'cough': True, 'sore_throat': False, 'headache': False, 'nausea': False, 'rash': False, 'shortness_of_breath': False}
  status: {'severity': 'mild', 'duration_days': 1}
```

RBR OUTPUT

```
------------------------------------------------------------
  RULE-BASED REASONING
------------------------------------------------------------

Rule-Based Reasoning Actions:
- No rule matched. Monitor your symptoms and rest.
```

CBR OUTPUT

```
------------------------------------------------------------
  CASE-BASED REASONING
------------------------------------------------------------
[Retrieve] Most similar past case found (86% similar)
          Past problem: ['cough', 'shortness_of_breath']

Proposed solution: Possible asthma/respiratory issue: use inhaler if prescribed and seek care.

Would you like to adjust this solution?
  [1] Keep it unchanged
  [2] Edit it manually
Your choice: 1

Final CBR Solution: Possible asthma/respiratory issue: use inhaler if prescribed and seek care.

[Retain] New case saved to case_base.json. Case base now has 6 cases.

Check another problem?
  [1] Yes
  [2] No, exit
Your choice: 2

Thank you. Remember: this is a demo, not medical advice.
```

Code

```
import json
import os

# PART 1 -- KNOWLEDGE REPRESENTATION (KR) SETUP

def ask_choice(prompt, options):
    """Show a numbered menu and keep asking until a valid option is chosen.
    `options` is a list of (label, value) tuples. Returns the chosen value."""
    print(f"\n{prompt}")
    for i, (label, _) in enumerate(options, start=1):
        print(f"  [{i}] {label}")
    while True:
        raw = input("Your choice: ").strip()
        if raw.isdigit() and 1 <= int(raw) <= len(options):
            return options[int(raw) - 1][1]
        print(f"  Invalid input. Please enter a number from 1 to
{len(options)}.")


def collect_facts():
    """Task 1A -- Build the knowledge base (facts dictionary) from the
```

```
    user's limited-option answers.

    Categories : vitals, symptoms, status
    Facts      : temperature, has_fever, cough, sore_throat, headache,
                 nausea, rash, shortness_of_breath, severity, duration_days
    """
    print("=" * 60)
    print("  MEDICAL SYMPTOM ADVISOR  (KR + Rule-Based + Case-Based)")
    print("=" * 60)

    problem = ask_choice(
        "What kind of medical problem are you experiencing?",
        [
            ("Fever / flu-like illness", "flu"),
            ("Cough or breathing difficulty", "respiratory"),
            ("Stomach / digestive problem", "digestive"),
            ("Headache", "headache"),
            ("Skin rash / allergy", "skin"),
        ],
    )

    # Start with a "nothing is wrong" baseline, then fill in from the answers.
    facts = {
        "vitals": {"temperature": 36.8, "has_fever": False},
        "symptoms": {
            "cough": False,
            "sore_throat": False,
            "headache": False,
            "nausea": False,
            "rash": False,
            "shortness_of_breath": False,
        },
        "status": {"severity": "mild", "duration_days": 1},
    }

    # --- primary problem sets the main symptoms ---
    if problem == "flu":
        facts["symptoms"]["headache"] = True
        facts["symptoms"]["sore_throat"] = ask_choice(
            "Do you have a sore throat?", [("Yes", True), ("No", False)])
        facts["symptoms"]["cough"] = ask_choice(
            "Do you have a cough?", [("Yes", True), ("No", False)])
    elif problem == "respiratory":
```

```
        facts["symptoms"]["cough"] = True
        facts["symptoms"]["shortness_of_breath"] = ask_choice(
            "Are you having trouble breathing?", [("Yes", True), ("No", False)])
        facts["symptoms"]["sore_throat"] = ask_choice(
            "Do you have a sore throat?", [("Yes", True), ("No", False)])
    elif problem == "digestive":
        facts["symptoms"]["nausea"] = True
    elif problem == "headache":
        facts["symptoms"]["headache"] = True
        facts["symptoms"]["nausea"] = ask_choice(
            "Do you also feel nauseous?", [("Yes", True), ("No", False)])
    elif problem == "skin":
        facts["symptoms"]["rash"] = True

    # --- vitals ---
    facts["vitals"]["temperature"] = ask_choice(
        "How is your body temperature?",
        [
            ("Normal (below 37.5 C)", 36.8),
            ("Mild fever (37.5 - 38.9 C)", 38.2),
            ("High fever (39 C or above)", 39.5),
        ],
    )
    facts["vitals"]["has_fever"] = facts["vitals"]["temperature"] >= 37.5

    # --- status ---
    facts["status"]["severity"] = ask_choice(
        "How severe are the symptoms?",
        [("Mild", "mild"), ("Moderate", "moderate"), ("Severe", "severe")],
    )
    facts["status"]["duration_days"] = ask_choice(
        "How long have you had these symptoms?",
        [("Less than 2 days", 1), ("2 to 5 days", 3), ("More than 5 days", 7)],
    )
    return facts

# PART 2 -- RULE-BASED REASONING (RBR)

# Task 2A -- IF-THEN rules (each returns an action, or None if not triggered)

def rule_fever(facts):
    if facts["vitals"]["has_fever"]:
        return "Take paracetamol and drink plenty of fluids"
```

```python
def rule_high_fever(facts):
    if facts["vitals"]["temperature"] >= 39:
        return "See a doctor within 24 hours (high fever)"

def rule_flu(facts):
    s = facts["symptoms"]
    if facts["vitals"]["has_fever"] and (s["cough"] or s["sore_throat"]):
        return "Rest at home, stay hydrated, and avoid close contact with others"

def rule_breathing_emergency(facts):
    if facts["symptoms"]["shortness_of_breath"]:
        return "EMERGENCY: Seek immediate medical care (difficulty breathing)"

def rule_digestive(facts):
    if facts["symptoms"]["nausea"]:
        return "Sip oral rehydration solution and eat bland food (rice, toast)"

def rule_headache(facts):
    s = facts["symptoms"]
    if s["headache"] and facts["status"]["severity"] == "mild":
        return "Rest in a quiet, dark room and drink water"

def rule_allergy(facts):
    if facts["symptoms"]["rash"] and not facts["vitals"]["has_fever"]:
        return "Take an antihistamine and avoid the suspected allergen"

def rule_severe(facts):
    if facts["status"]["severity"] == "severe":
        return "Visit a clinic or hospital today (severe symptoms)"

def rule_prolonged(facts):
    if facts["status"]["duration_days"] > 5:
        return "Consult a physician (symptoms lasting more than 5 days)"


RULES = [
    rule_fever, rule_high_fever, rule_flu, rule_breathing_emergency,
    rule_digestive, rule_headache, rule_allergy, rule_severe, rule_prolonged,
]

# Task 2B -- Read the knowledge base, check each rule, and return all the actions
that were inferred.
```

```
def inference_engine(facts, rules):
    actions = []
    for rule in rules:
        result = rule(facts)
        if result is not None:
            actions.append(result)
    return actions


def display_rbr_output(actions):
    """Task 2C -- Display output."""
    print("\nRule-Based Reasoning Actions:")
    if actions:
        for action in actions:
            print(f"- {action}")
    else:
        print("- No rule matched. Monitor your symptoms and rest.")

# PART 3 -- CASE-BASED REASONING (CBR)

# Task 3A -- Starting case base (problem features + matching solution).
DEFAULT_CASES = [
    {"problem": {"fever": True,  "cough": True,  "sore_throat": True,
 "headache": True,
                 "nausea": False, "rash": False, "shortness_of_breath": False},
     "solution": "Likely influenza: rest, fluids, and paracetamol."},
    {"problem": {"fever": False, "cough": True,  "sore_throat": False,
"headache": False,
                 "nausea": False, "rash": False, "shortness_of_breath": True},
     "solution": "Possible asthma/respiratory issue: use inhaler if prescribed
and seek care."},
    {"problem": {"fever": False, "cough": False, "sore_throat": False,
"headache": False,
                 "nausea": True,  "rash": False, "shortness_of_breath": False},
     "solution": "Likely stomach upset: oral rehydration and bland diet."},
    {"problem": {"fever": False, "cough": False, "sore_throat": False,
"headache": True,
                 "nausea": True,  "rash": False, "shortness_of_breath": False},
     "solution": "Possible migraine: rest in a dark room and take a pain
reliever."},
    {"problem": {"fever": False, "cough": False, "sore_throat": False,
"headache": False,
                 "nausea": False, "rash": True,  "shortness_of_breath": False},
```

```
     "solution": "Likely allergic reaction: antihistamine and avoid the
allergen."},
]

# The JSON file is stored in the same folder as this script.
CASE_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)),
"case_base.json")

# Write the whole case base to the JSON file.
def save_case_base(cases):
    try:
        with open(CASE_FILE, "w", encoding="utf-8") as f:
            json.dump(cases, f, indent=4)
    except OSError as e:
        print(f"[Warning] Could not save cases to file: {e}")

# Load cases from the JSON file. If the file is missing or damaged, start from
the default cases (and create the file).
def load_case_base():
    if os.path.exists(CASE_FILE):
        try:
            with open(CASE_FILE, "r", encoding="utf-8") as f:
                cases = json.load(f)
            if (isinstance(cases, list) and cases
                    and all("problem" in c and "solution" in c for c in cases)):
                print(f"[Case base] Loaded {len(cases)} cases from {CASE_FILE}")
                return cases
            print("[Warning] case_base.json has an invalid format. Using default
cases.")
        except (OSError, json.JSONDecodeError) as e:
            print(f"[Warning] Could not read case_base.json ({e}). Using default
cases.")
    cases = [dict(c) for c in DEFAULT_CASES]
    save_case_base(cases)
    return cases


case_base = load_case_base()

# Convert the knowledge base into a CBR problem description.
def facts_to_problem(facts):
    s = facts["symptoms"]
    return {
```

```
        "fever": facts["vitals"]["has_fever"],
        "cough": s["cough"],
        "sore_throat": s["sore_throat"],
        "headache": s["headache"],
        "nausea": s["nausea"],
        "rash": s["rash"],
        "shortness_of_breath": s["shortness_of_breath"],
    }

# Task 3B -- Fraction of features that have the same value.
def similarity(new_problem, old_problem):
    keys = set(new_problem) | set(old_problem)
    matches = sum(1 for k in keys if new_problem.get(k) == old_problem.get(k))
    return matches / len(keys)

# Task 3B -- Find the most similar past case.
def retrieve(new_problem, cases):
    best_case, best_score = None, -1.0
    for case in cases:
        score = similarity(new_problem, case["problem"])
        if score > best_score:
            best_case, best_score = case, score
    return best_case, best_score

# Task 3C -- Return the solution of the most similar case.
def reuse(best_case):

    return best_case["solution"]

# Task 3D -- Let the user adjust the solution or keep it unchanged.
def revise(solution):
    print(f"\nProposed solution: {solution}")
    choice = ask_choice(
        "Would you like to adjust this solution?",
        [("Keep it unchanged", False), ("Edit it manually", True)],
    )
    if choice:
        new_text = input("Type your revised solution: ").strip()
        if new_text:
            return new_text
    return solution

#3E - Add the new case to the case base
```

```
def retain(case_base, new_problem, final_solution):
    case_base.append({"problem": new_problem, "solution": final_solution})
    save_case_base(case_base)
    print(f"\n[Retain] New case saved to case_base.json. "
          f"Case base now has {len(case_base)} cases.")


def run_cbr(facts):
    print("\n" + "-" * 60)
    print("  CASE-BASED REASONING")
    print("-" * 60)
    new_problem = facts_to_problem(facts)

    best_case, score = retrieve(new_problem, case_base)
    print(f"[Retrieve] Most similar past case found ({score:.0%} similar)")
    print(f"           Past problem: "
          f"{[k for k, v in best_case['problem'].items() if v] or ['no
symptoms']}")

    solution = reuse(best_case)
    final = revise(solution)
    print(f"\nFinal CBR Solution: {final}")
    retain(case_base, new_problem, final)


# MAIN PROGRAM

def main():
    while True:
        facts = collect_facts()

        print("\nKnowledge Base (facts):")
        for category, items in facts.items():
            print(f"  {category}: {items}")

        actions = inference_engine(facts, RULES)
        print("\n" + "-" * 60)
        print("  RULE-BASED REASONING")
        print("-" * 60)
        display_rbr_output(actions)

        run_cbr(facts)
```

```
        again = ask_choice("Check another problem?",
                           [("Yes", True), ("No, exit", False)])
        if not again:
            print("\nThank you. Remember: this is a demo, not medical advice.")
            break


if __name__ == "__main__":
    main()
```